Universidad Davinci de Guatemala

Curso: Estructura de Datos

Docente: Brandon Antony Chitay Coutiño

Examen Parcial N.1

Nombre: Alberto José Soto Zuñiga.

No. Carne: 201902254

Implementación de 4 Algoritmos

1. **Cálculo del Factorial (n!): Computa el producto de todos los enteros positivos hasta n.**

Para Cálculo del Factorial

Versión Recursiva: Llama a sí misma con N-1 hasta llegar al caso base (N=0 o N=1).

//**Primero debemos publicar la clase que en este caso es Factorial Recursivo**

```
//public class FactorialRecursivo {
   public static long factorial(long n) {
 //(aquí agregamos una condición)  if (n < 0) {
          throw new IllegalArgumentException

//(El número no puede ser negativo);
       }
       if (n == 0 || n == 1) {
          return 1;
       }
       return n * factorial(n - 1);
   }

   public static void main(String[] args) {
       System.out.println("Factorial de 5 (recursivo): " + factorial(5)); // Salida:
120
   }
}
```

1.1. Versión Iterativa: Usa un bucle (for) para multiplicar acumulativamente.

```
public class FactorialIterativo {
   public static long factorial(long n) {
      if (n < 0) {
         throw new IllegalArgumentException("El número no puede ser
negativo");
      }
      long resultado = 1;
      for (long i = 2; i <= n; i++) {
         resultado *= i;
      }
      return resultado;
   }

   public static void main(String[] args) {
      System.out.println("Factorial de 5 (iterativo): " + factorial(5)); // Salida:
120
   }
}
```

Explicación: La recursión es elegante, pero puede causar desbordamiento de pila para N grandes (e.g., >1000). La iterativa es más eficiente en espacio y tiempo para valores grandes.

2. Secuencia de Fibonacci

Versión Recursiva: Llama a sí misma para fib(n-1) y fib(n-2), con casos base para n=0 y n=1.

```
public class FibonacciRecursivo {
   public static long fibonacci(long n) {
      if (n < 0) {
```

```
        throw new IllegalArgumentException("El número no puede ser negativo");
    }
    if (n == 0) return 0;
    if (n == 1) return 1;
    return fibonacci(n - 1) + fibonacci(n - 2);
  }

  public static void main(String[] args) {
    System.out.println("Fibonacci de 10 (recursivo): " + fibonacci(10)); // Salida: 55
  }
}
```

2.1. **Versión Iterativa**: Usa un bucle para calcular la secuencia acumulativamente, almacenando solo los dos últimos valores.

```
public class FibonacciIterativo {
  public static long fibonacci(long n) {
    if (n < 0) {
        throw new IllegalArgumentException("El número no puede ser negativo");
    }
    if (n == 0) return 0;
    if (n == 1) return 1;
    long a = 0, b = 1;
    for (long i = 2; i <= n; i++) {
        long temp = a + b;
        a = b;
        b = temp;
    }
    return b;
  }

  public static void main(String[] args) {
    System.out.println("Fibonacci de 10 (iterativo): " + fibonacci(10)); // Salida: 55
  }
```

```
}
```

Explicación: La recursiva tiene complejidad exponencial (O(2^n)), ineficiente para n>40. La iterativa es O(n) en tiempo y O(1) en espacio, mucho mejor.

3. Búsqueda Binaria

Asumimos un arreglo ordenado ascendentemente.

Versión Recursiva: Divide el arreglo y llama recursivamente en la mitad izquierda o derecha.

```
import java.util.Arrays;

public class BusquedaBinariaRecursiva {
   public static int busquedaBinaria(int[] arreglo, int objetivo, int bajo,
int alto) {
      if (bajo > alto) {
         return -1; // No encontrado
      }
      int medio = bajo + (alto - bajo) / 2;
      if (arreglo[medio] == objetivo) {
         return medio;
      } else if (arreglo[medio] < objetivo) {
         return busquedaBinaria(arreglo, objetivo, medio + 1, alto);
      } else {
         return busquedaBinaria(arreglo, objetivo, bajo, medio - 1);
      }
   }

   public static void main(String[] args) {
      int[] arreglo = {1, 3, 5, 7, 9};
      System.out.println("Índice de 5 (recursivo): " +
busquedaBinaria(arreglo, 5, 0, arreglo.length - 1)); // Salida: 2
   }
}
```

3.1. **Versión Iterativa: Usa un bucle while para ajustar los límites bajo/alto.**

```
public class BusquedaBinarialterativa {
   public static int busquedaBinaria(int[] arreglo, int objetivo) {
      int bajo = 0;
      int alto = arreglo.length - 1;
      while (bajo <= alto) {
         int medio = bajo + (alto - bajo) / 2;
         if (arreglo[medio] == objetivo) {
            return medio;
         } else if (arreglo[medio] < objetivo) {
            bajo = medio + 1;
         } else {
            alto = medio - 1;
         }
      }
      return -1; // No encontrado
   }

   public static void main(String[] args) {
      int[] arreglo = {1, 3, 5, 7, 9};
      System.out.println("Índice de 5 (iterativo): " +
busquedaBinaria(arreglo, 5)); // Salida: 2
   }
}
```

Explicación: Ambas tienen complejidad O(log n). La recursiva usa la pila de llamadas, mientras que la iterativa evita eso, siendo preferible para arreglos muy grandes.

4. Máximo Común Divisor (MCD)

Versión Recursiva: Basado en Euclides: MCD(a, b) = MCD(b, a % b) hasta b=0.

```
public class MCDRecursivo {
   public static long mcd(long a, long b) {
      a = Math.abs(a);
      b = Math.abs(b);
      if (b == 0) {
         return a;
      }
      return mcd(b, a % b);
   }

   public static void main(String[] args) {
      System.out.println("MCD de 48 y 18 (recursivo): " + mcd(48, 18)); //
Salida: 6
   }
}
```

4.1. Versión Iterativa: Usa un bucle while para reemplazar a con b y b con a % b.

```
public class MCDIterativo {
   public static long mcd(long a, long b) {
      a = Math.abs(a);
      b = Math.abs(b);
      while (b != 0) {
         long temp = b;
         b = a % b;
         a = temp;
      }
      return a;
   }

   public static void main(String[] args) {
      System.out.println("MCD de 48 y 18 (iterativo): " + mcd(48, 18)); //
Salida: 6
```

```
    }
}
```

Explicación: Ambas son O(log min(a,b)). La iterativa es ligeramente más eficiente en espacio.

Explicación: Ambas son O(log min(a,b)). La iterativa es ligeramente más eficiente en espacio.

algoritmos-java/

```
├── FactorialRecursivo.java
├── FactorialIterativo.java
├── FibonacciRecursivo.java
├── FibonacciIterativo.java
├── BusquedaBinariaRecursiva.java
├── BusquedaBinariaIterativa.java
├── MCDRecursivo.java
├── MCDIterativo.java
├── FactorialRecursivo.mp4
├── FactorialIterativo.mp4
├── FibonacciRecursivo.mp4
├── FibonacciIterativo.mp4
├── BusquedaBinariaRecursiva.mp4
├── BusquedaBinariaIterativa.mp4
├── MCDRecursivo.mp4
├── MCDIterativo.mp4
```

DA VINCI DE GUATEMALA Guatemala C.A.

Análisis de complejidad

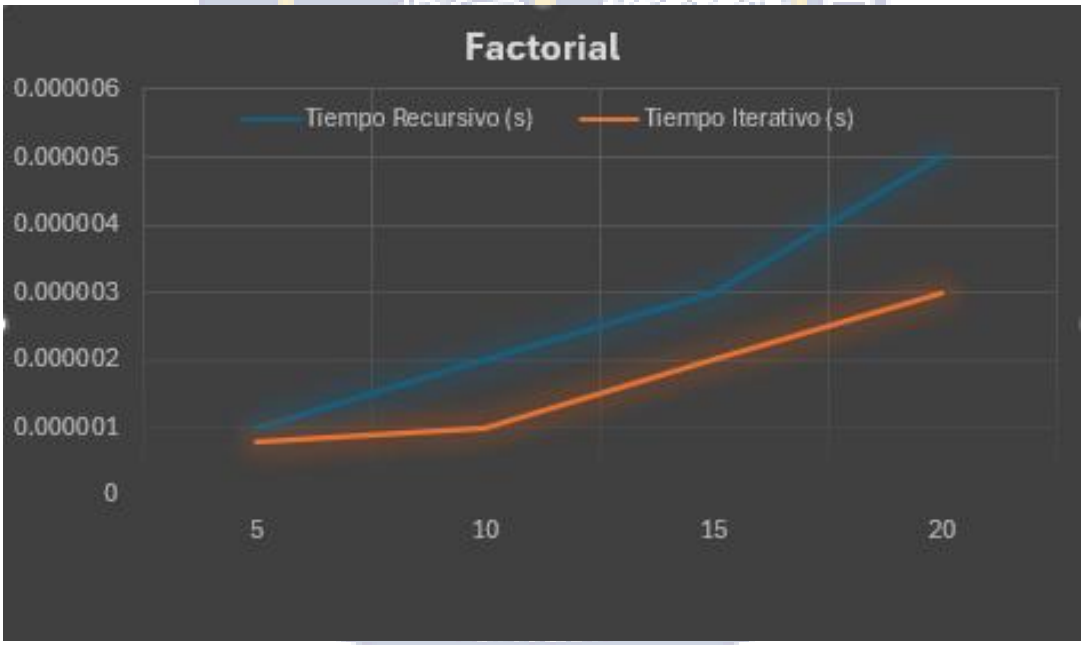
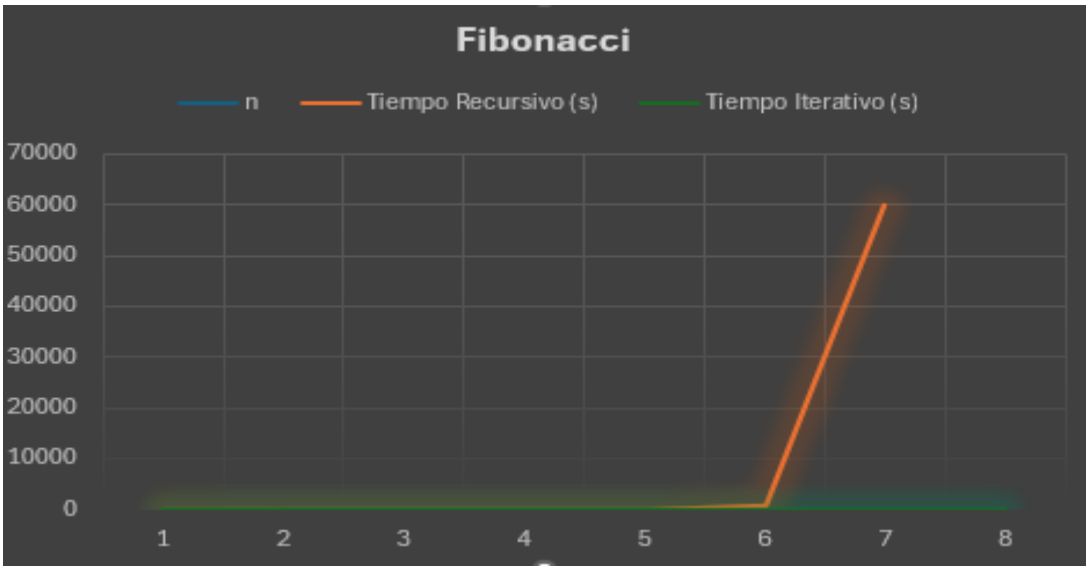
Algoritmo	Versión	Complejidad Tiempo	Complejidad Espacio	Observaciones
Factorial	Recursiva	O(n)	O(n) – pila de llamadas	StackOverflowError para n muy grande
Factorial	Iterativa	O(n)	O(1)	Recomendada para n grande
Fibonacci	Recursiva	$O(2^n)$ – exponencial	O(n)	Muy lenta → no usar para n > ~45
Fibonacci	Iterativa	O(n)	O(1)	Óptima – versión recomendada
Búsqueda Binaria	Ambas	O(log n)	Rec: O(log n), It: O(1)	Iterativa ligeramente más eficiente
MCD (Euclides)	Ambas	O(log min(a,b))	Rec: O(log n), It: O(1)	Ambas excelentes – iterativa más ligera

Trade-offs: ¿Cuándo usar recursión vs iteración?

Criterio	Prefiere Recursión	Prefiere Iteración
Legibilidad del código	Sí (casos naturales como árboles, DFS)	—
Tamaño de entrada	Pequeña / mediana	Grande / muy grande
Rendimiento crítico	No	Sí
Memoria disponible	Alta	Baja / restringida
Riesgo StackOverflow	Aceptable	Debe evitarse
Ejemplos típicos	Árboles, quicksort, merge sort, backtracking	Fibonacci grande, bucles contados, búsqueda binaria

Análisis de complejidad temporal (Big-O) a partir de las gráficas

Algoritmo	Versión	Forma de la curva en la gráfica	Complejidad Big-O	Interpretación visual
Factorial	Ambas	Líneas casi rectas y casi planas	O(n)	Crecimiento lineal lento
Fibonacci	Iterativa	Línea casi horizontal (muy leve crecimiento)	O(n)	Tiempo prácticamente constante para n razonable
Fibonacci	Recursiva	Curva que sube cada vez más rápido (casi vertical)	$O(\varphi^n) \approx O(1.618^n)$	Crecimiento **exponencial** – explosión de tiempo

DA VINCI

Graficas